

Membedah Aplikasi "CariMakan": Panduan Analogi Komponen React untuk Pemula

Selamat datang, calon Arsitek Perangkat Lunak. Dalam membangun aplikasi "CariMakan", kita tidak sekadar menulis baris kode; kita sedang merancang sebuah sistem yang terukur. Sebagai pengembang, tugas Anda adalah beralih dari pola pikir "penulis skrip" menjadi seorang perancang struktur. Panduan ini akan membedah anatomi aplikasi kita menggunakan analogi dunia nyata untuk memastikan fondasi kognitif Anda sekuat kode yang Anda tulis.

1. Pendahuluan: Mengapa Kita Memecah UI?

Bayangkan Anda diminta membangun sebuah gedung pencakar langit. Apakah Anda akan mencetak seluruh gedung dari satu bongkahan beton raksasa? Tentu tidak. Anda akan menggunakan modul prefabrikasi: jendela, pintu, dan panel dinding yang terstandarisasi. Inilah esensi dari **Arsitektur Komponen** dalam React.

Dalam aplikasi "CariMakan", kita menolak menulis satu file HTML raksasa yang berantakan (sering disebut sebagai *Spaghetti Code*). Kita memecah antarmuka menjadi potongan-potongan kecil yang mandiri karena tiga alasan arsitektural utama:

- **Reusability (Penggunaan Kembali):** Sama seperti satu desain kusen jendela yang bisa dipasang di seratus kamar, satu komponen FoodCard yang kita buat bisa digunakan untuk menampilkan ribuan menu berbeda tanpa menulis ulang kode.
- **Scalability (Skalabilitas):** Ketika aplikasi "CariMakan" berkembang dari sekadar daftar menu menjadi sistem reservasi yang kompleks, kita cukup menambah komponen baru tanpa merusak integritas struktur yang sudah ada.
- **Simplicity (Kesederhanaan):** Melacak *bug* di dalam komponen SearchBar yang hanya berisi 20 baris kode jauh lebih efisien daripada mencarinya di antara ribuan baris kode dalam satu file tunggal.

Memahami pemecahan ini adalah langkah pertama transisi Anda dari seorang pemula menjadi arsitek UI yang handal.

2. Cetak Biru "CariMakan": JSX dan Struktur Folder

Sebelum konstruksi dimulai, seorang arsitek membutuhkan lahan yang jelas dan denah yang akurat. Dalam ekosistem React, kita menggunakan Vite sebagai pengelola lahan kita.

Perintah `npm create vite@latest` adalah proses "**Menyiapkan Lahan Konstruksi**". Vite memagari lahan, menyiapkan aliran listrik (server lokal), dan menyediakan peralatan dasar. Namun, lahan tersebut seringkali masih memiliki "semak belukar" berupa file *boilerplate* bawaan yang tidak kita perlukan. Tugas pertama Anda sebagai arsitek adalah membersihkan lahan tersebut agar kita memiliki kanvas kosong yang siap bangun.

Tabel Perbandingan: Konstruksi vs. Dunia React

Konsep Dunia Nyata	Padanan di React	Fungsi dalam Proyek "CariMakan"

Cetak Biru (Denah)	JSX	Menentukan struktur visual (tombol, gambar, teks) menggunakan sintaks mirip HTML.
Kotak Perkakas	src/components	Folder sentral tempat kita menyimpan semua "alat" atau komponen yang telah kita fabrikasi.
Pembersihan Lahan	Hapus Boilerplate	Proses membuang file bawaan Vite agar struktur proyek tetap bersih dan efisien.
Izin Bangunan & Alat Berat	Vite & NPM	Infrastruktur yang menjalankan, mengawasi, dan membungkus kode kita menjadi aplikasi nyata.

Setelah lahan bersih dan denah dipahami, kita bisa mulai memasang elemen-elemen struktur pertama yang akan membentuk kerangka aplikasi kita.

3. Komponen Fondasi: Header, SearchBar, dan Footer

Jika aplikasi "CariMakan" adalah sebuah restoran fisik, maka komponen-komponen ini adalah elemen interior tetapnya.

1. **Header:** Ini adalah papan nama di depan toko. Ia memberikan identitas visual dan konteks instan kepada pengunjung tentang di mana mereka berada.
2. **SearchBar:** Berbeda dengan dinding yang pasif, SearchBar adalah "Pusat Interaksi" —seperti meja resepsionis tempat tamu memberikan instruksi tentang apa yang mereka cari.
3. **Footer:** Ini adalah papan informasi di area keluar yang memberikan rasa tuntas pada pengalaman pengguna, berisi detail kontak atau hak cipta.

Catatan Arsitek: Sangat krusial untuk memastikan folder `src/components` Anda terorganisir sejak awal. Membangun komponen statis ini adalah cara kita memvalidasi bahwa "aliran listrik" dan "fondasi" aplikasi kita sudah terpasang dengan benar sebelum kita memasukkan data yang dinamis.

4. FoodCard & Mapping: Cetakan Kue yang Cerdas

Dalam rekayasa perangkat lunak, kita memegang teguh prinsip **DRY (Don't Repeat Yourself)**. Seorang pemula mungkin akan membuat 100 file untuk 100 menu makanan, tetapi seorang arsitek akan membuat satu "**Cetakan Kue**".

Komponen FoodCard adalah cetakan tersebut. Kita mendefinisikan strukturnya sekali (tempat gambar, posisi nama makanan, letak harga), lalu kita menggunakan fungsi `.map()` sebagai ban berjalan di pabrik kita.

- **Bahan Baku (Mock Data):** Array berisi informasi mentah tentang makanan.
- **Proses Cetak (.map):** Fungsi ini mengambil satu per satu data dari "Bahan Baku", memasukkannya ke dalam cetakan FoodCard, dan menyajikannya secara otomatis di layar.

Dengan pendekatan ini, Anda bisa mengelola ribuan menu hanya dengan memelihara satu file komponen. Inilah efisiensi tingkat tinggi dalam pengembangan UI.

5. Dinamika Aliran Data: Props vs. State

Aplikasi "CariMakan" mulai menjadi "hidup" saat data mulai mengalir. Memahami perbedaan antara Props dan State adalah kunci untuk mengendalikan logika aplikasi Anda.

Tabel Perbandingan Pengelolaan Data

Fitur	Props (Nota Pesanan)	State (Ingatan Jangka Pendek)
Sumber	Diberikan oleh "Atasan" (Parent)	Dikelola secara internal oleh komponen
Sifat	<i>Immutable</i> (Hanya baca)	<i>Mutable</i> (Bisa diubah sendiri)
Tujuan	Menentukan spesifikasi apa yang harus ditampilkan FoodCard	Mengingat apa yang diketik pengguna di SearchBar

Logika Penyaringan: Dalam Sesi 2, kita menghubungkan keduanya. Ketika pengguna mengetik di SearchBar, ia mengubah **State**. State ini kemudian memberi tahu fungsi Mapping untuk hanya menggunakan "Cetakan Kue" pada data yang sesuai dengan kata kunci tersebut.

Insight Troubleshooting: State bersifat *asynchronous*. Analoginya seperti **Pelayan Restoran**: Saat Anda memberikan pesanan (setState), pelayan butuh waktu untuk mencatatnya sebelum pesanan itu sampai ke dapur. Jika Anda mencoba mengecek pesanan tepat satu milidetik setelah pelayan pergi (console.log), pelayan tersebut mungkin belum selesai mencatat. Inilah mengapa nilai state sering terlihat "terlambat" dalam log pencarian Anda.

6. Jembatan ke Dunia Luar: API & useEffect

Restoran yang hebat tidak hanya mengandalkan stok di gudang sendiri; ia harus terhubung dengan supplier besar. Dalam "CariMakan", supplier kita adalah **TheMealDB (API)**.

Jika API adalah suppliernya, maka useEffect adalah **Manajer Toko**. Manajer inilah yang bertugas memutuskan kapan harus menelepon supplier untuk meminta kiriman data.

Alur Pengiriman Data:

1. **Mounting (Toko Dibuka):** Saat komponen pertama kali muncul, useEffect memicu panggilan ke API.
2. **Loading State (Papan Pengumuman):** Sambil menunggu truk supplier datang, kita memasang papan pengumuman "Sedang Menunggu Stok" (Loading Spinner) agar pelanggan tidak bingung melihat rak kosong.
3. **Fetch & Update:** Begitu data sampai, Manajer (useEffect) memasukkannya ke dalam State, yang secara otomatis memicu "Cetakan Kue" (FoodCard) untuk memproduksi tampilan menu di layar secara instan.

7. Kesimpulan: Menjadi Arsitek UI yang Handal

Perjalanan Anda dari Sesi 1 hingga Sesi 3 telah membangun fondasi yang kokoh dalam memahami rekayasa UI modern. Anda telah belajar cara menyiapkan lahan dengan Vite, membangun struktur

dengan komponen, mengotomasi produksi dengan Mapping, hingga mengelola logistik data melalui API.

Tiga Poin Refleksi Utama:

1. **Modularitas adalah Kekuatan:** Dengan memecah UI, Anda membangun sistem yang mudah dirawat.
2. **Data adalah Aliran:** Mahir membedakan Props dan State adalah pembeda antara pengembang amatir dan profesional.
3. **Konektivitas adalah Kunci:** Aplikasi modern adalah jembatan antara pengguna dan sumber data global (API).

Kini, kerangka bangunan aplikasi "CariMakan" Anda telah berdiri tegak dan fungsional. Anda telah memiliki struktur yang benar dan aliran data yang lancar. Langkah Anda selanjutnya adalah mempercantik tampilan dengan **Styling** dan mengatur navigasi ruangan dengan **Routing**. Teruslah membangun dengan ketelitian dan kepercayaan diri seorang arsitek!